

Savorealo — Visión General del Proyecto

Savorealo es una red social gastronómica para descubrir, guardar, publicar y cocinar recetas, con experiencias sociales completas: perfiles, seguidores, comentarios, mensajes directos, lugares y generación de recetas con inteligencia artificial.

Los tres pilares del proyecto

	Frontend Web	Backend API	Mobile Android
Repo	<code>savorealo/savorealo_frontend</code>	<code>savorealo/api</code>	<code>acanojiDev/TFG-LET-ME-COOK-MOBILE</code>
Stack	Angular 21 · SSR · PWA	Cloudflare Workers · GraphQL	Kotlin 2.2 · Jetpack Compose
Datos	Apollo Angular · Supabase JS	Prisma 7 · PostgreSQL	Retrofit · Supabase KT
Deploy	Cloudflare Pages	Cloudflare Workers	Google Play / APK

Arquitectura del sistema

```
flowchart TD
    U(["👤 Usuario"])

    subgraph Clientes
        A["🌐 Angular 21\nSSR / PWA"]
        M["📱 Android Kotlin\nJetpack Compose"]
    end

    subgraph Backend
        W["⚡ Cloudflare Worker\nGraphQL Yoga + Pothos"]
    end

    subgraph Servicios
        SB["🗄️ Supabase\nAuth · Storage · Realtime"]
        DB["🐘 PostgreSQL\nPrisma 7 + Accelerate"]
    end

    U --> A
    U --> M
    A -->|"Apollo GraphQL\nPOST /graphql"| W
    M -->|"Retrofit\nPOST /graphql"| W
    A <-->|"Auth / Storage"| SB
    M <-->|"Auth / Realtime"| SB
    W -->|"Verifica JWT"| SB
    W <-->|"Prisma 7"| DB
```

Flujo de autenticación

```
sequenceDiagram
    actor U as Usuario
    participant C as Cliente (Web/Mobile)
    participant S as Supabase Auth
    participant W as Worker GraphQL
    participant DB as PostgreSQL

    U->>C: Login (email + contraseña)
    C->>S: supabase.auth.signIn(...)
    S-->>C: JWT + refresh_token
    Note over C: Token guardado en localStorage / persistencia KT
    C->>W: POST /graphql + Authorization: Bearer JWT
    W->>W: verifyJWT(token, SUPABASE_JWT_SECRET)
    W->>DB: Prisma query/mutation
    DB-->>W: Datos de dominio
    W-->>C: Respuesta GraphQL
```

Stack tecnológico completo

Capa	Frontend Web	Backend	Mobile Android
Lenguaje	TypeScript	TypeScript	Kotlin 2.2
Framework	Angular 21	—	Jetpack Compose
API Client	Apollo Angular	—	Retrofit + OkHttp
Auth	Supabase JS	JWT HMAC-SHA256	Supabase KT
UI	PrimeNG 21 + Tailwind 3	—	Material 3
Estado	Signal Stores	—	ViewModel + StateFlow
Persistencia	—	Prisma 7 + Accelerate	Room
Tiempo real	Supabase Realtime	—	Supabase Realtime
Tests	Vitest	Vitest	JUnit + Espresso
Calidad	ESLint + Prettier + Husky	ESLint	Detekt
Deploy	Cloudflare Pages	Cloudflare Workers	APK / Google Play

Estructura del repositorio

```
tfg/  
├─ savorealo/           ← Documentación y diagramas del proyecto  
├─ savorealo_frontend/  
│   └─ src/app/  
│       ├─ core/        ← Guards, interceptors, repos, services, store  
│       ├─ features/     ← Páginas por dominio (lazy-loaded)  
│       ├─ graphql/      ← Queries y mutations Apollo  
│       └─ shared/       ← Componentes, pipes, directivas reutilizables  
├─ api/                 ← API GraphQL en Cloudflare Workers  
│   └─ src/  
│       ├─ index.ts      ← Entry point, CORS, auth, routing  
│       ├─ lib/          ← Auth, Prisma, manejo de errores  
│       └─ schema/       ← Schema Pothos organizado por dominio  
└─ TFG-LET-ME-COOK-MOBILE/ ← App Android nativa  
    └─ app/src/main/java/es/PapayaSA/letmecook/  
        ├─ data/         ← Models, remote APIs, repositories, Room  
        ├─ di/           ← Módulos Hilt  
        ├─ ui/           ← Screens Compose, navigation, theme  
        └─ worker/       ← WorkManager tasks
```

URLs del proyecto

Entorno	URL
Frontend producción	https://app.savorealo.com
API producción	https://develop.app.savorealo.com/api/graphql
Health check	https://develop.app.savorealo.com/api/health
Frontend local	http://localhost:4200
API local	http://localhost:8787/graphql

Instalación rápida

```
# Frontend
git clone https://github.com/savorealo/savorealo_frontend.git
cd savorealo_frontend && npm install && npm start

# Backend
git clone https://github.com/savorealo/api.git
cd api && npm install && npx wrangler dev

# Mobile (Android Studio o CLI)
git clone https://github.com/acanojiDev/TFG-LET-ME-COOK-MOBILE.git
cd TFG-LET-ME-COOK-MOBILE && ./gradlew installDebug
```

Índice de documentación

Documento	Descripción
Frontend Web	Angular 21, rutas, patrón Repository, Signal Stores, i18n
Backend API	GraphQL, Cloudflare Workers, todas las operaciones
Mobile Android	Kotlin, Compose, Remote vs Repository, pantallas
Flujos del sistema	Auth, feed, IA, chat, CI/CD y más

Frontend Web — Angular 21

Aplicación web con **Angular 21**, renderizado del lado del servidor (SSR), detección de cambios **zoneless**, comunicación GraphQL vía **Apollo Angular**, UI con **PrimeNG 21** + **Tailwind CSS 3** y soporte **PWA** completo.

Comandos principales

```
npm start                # Dev server → http://localhost:4200
npm run build             # Build de producción
npm run build:cloudflare  # Build para Cloudflare Pages
npm test                 # Tests con Vitest
npm run test:cov         # Tests con cobertura
npm run lint             # ESLint --fix
npm run format           # Prettier --write
npm run compodoc:serve   # Documentación con Compodoc

ng generate component features/mi-feature/mi-componente
```


Stack

Capa	Tecnología	Detalle
Framework	Angular 21	Zoneless, SSR con <code>@angular/ssr</code>
API	Apollo Angular	GraphQL, queries tipadas
Auth / Storage	Supabase JS	Sesión, almacenamiento de imágenes
UI Components	PrimeNG 21	Preset Aura, paleta naranja customizada
Estilos	Tailwind CSS 3	Utility-first, dark mode vía <code>data-theme</code>
Estado	Signal Stores	<code>signal()</code> , <code>computed()</code> , zoneless
Testing	Vitest	Unitario + cobertura
Calidad	ESLint + Prettier + Husky	lint-staged en pre-commit

Aliases de path

Alias	Ruta real
<code>@core/*</code>	<code>src/app/core/*</code>
<code>@shared/*</code>	<code>src/app/shared/*</code>
<code>@features/*</code>	<code>src/app/features/*</code>
<code>@graphql/*</code>	<code>src/app/graphql/*</code>
<code>@env/*</code>	<code>src/environments/*</code>
<code>@testing/*</code>	<code>src/testing/*</code>

Arquitectura de capas

```
flowchart TD
    subgraph UI["Capa UI (features/)"]
        Pages["Páginas\n(lazy-loaded)"]
        Shared["Componentes compartidos\n(post-card, topbar, avatar...)"]
    end

    subgraph State["Capa de Estado (store/)"]
        Stores["Signal Stores\n(FeedStore, AuthStore, ChatStore...)"]
    end

    subgraph Domain["Capa de Dominio (services/)"]
        Services["Servicios\n(PostService, UserService...)"]
    end

    subgraph Data["Capa de Datos (repositories/)"]
        Repos["Interfaces IRepository\n(IPostRepository, IUserRepository...)"]
        GQL["Impl. GraphQL\n(PostGraphQLRepository)"]
        SBRepo["Impl. Supabase\n(UserSupabaseRepository)"]
    end

    subgraph External["Externo"]
        API["API GraphQL\n(Cloudflare Worker)"]
        SB["Supabase\n(Auth / Storage)"]
    end

    Pages --> Stores
    Stores --> Services
    Services --> Repos
    Repos --> GQL --> API
    Repos --> SBRepo --> SB
```

Estructura de carpetas

```
src/app/
├─ core/
│   ├─ guards/          authGuard, guestGuard (canActivate + canMatch)
│   ├─ interceptors/    auth (añade JWT), metrics
│   ├─ models/          interfaces de dominio (Post, User, Recipe...)
│   ├─ repositories/    interfaces + implementaciones concretas
│   ├─ services/        lógica de negocio (TranslationService, ContentTranslationS
│   ├─ store/           Signal Stores por dominio
│   ├─ tokens/          InjectionTokens (ENVIRONMENT, POST_REPOSITORY...)
│   └─ utils/           funciones puras
├─ features/            páginas lazy-loaded por dominio
├─ graphql/             queries y mutations Apollo tipadas
└─ shared/
    ├─ components/      app-shell, topbar, post-card, avatar, comment-section...
    ├─ directives/      click-outside, img-fallback
    └─ pipes/           translate, time-ago, truncate, message-markdown
```

Rutas y páginas

```
flowchart LR
    Auth["/auth\nLogin / Registro"]

    subgraph App ["App (autenticado)"]
        Feed["/\nFeed"]
        Explore["/explore\nExplorar"]
        AI["/ai\nGenerador IA"]
        Saved["/saved\nGuardados"]
        Chat["/chat\nMensajes"]
        Profile["/profile\nPerfil propio"]
        PubProfile["/profile/:username\nPerfil público"]
        Settings["/settings\nAjustes"]
        Post["/post/:id\nDetalle post"]
        Cook["/cook/:id\nModo cocina"]
        Shopping["/shopping\nLista compra"]
        Places["/places\nLugares"]
        PlaceDetail["/places/:id\nDetalle lugar"]
    end

    Auth --> Feed
```

Ruta	Página	Acceso
<code>/auth</code>	Login y registro	Público
<code>/</code>	Feed principal	Autenticado
<code>/explore</code>	Exploración de contenido	Autenticado
<code>/ai</code>	Generador de recetas con IA	Autenticado
<code>/saved</code>	Publicaciones guardadas	Autenticado
<code>/chat</code>	Mensajes directos	Autenticado
<code>/profile</code>	Perfil propio	Autenticado
<code>/profile/:username</code>	Perfil público	Autenticado
<code>/settings</code>	Ajustes de cuenta	Autenticado
<code>/post/:id</code>	Detalle de publicación	Autenticado
<code>/cook/:id</code>	Modo cocina (paso a paso)	Autenticado
<code>/shopping</code>	Lista de la compra	Autenticado
<code>/places</code>	Lugares gastronómicos	Autenticado
<code>/places/:id</code>	Detalle de lugar	Autenticado

Patrón Repository

Cada dominio tiene una interfaz, una o varias implementaciones y un token de inyección. Los servicios nunca dependen de la implementación concreta.

```

flowchart LR
    Service["PostService\n(inyecta por token)"]
    Token["POST_REPOSITORY\n(InjectionToken)"]
    Interface["IPostRepository\n(interfaz)"]
    GQL["PostGraphQLRepository"]
    SB["PostSupabaseRepository"]
    Config["app.config.ts\nprovide: POST_REPOSITORY\nuseClass: PostGraphQLRepository"]

    Service -->|inject| Token
    Token --> Interface
    Interface --> GQL
    Interface --> SB
    Config -.→|binding|. Token

```

```

// Servicio - solo conoce la interfaz
constructor(@Inject(POST_REPOSITORY) private repo: IPostRepository) {}

// app.config.ts - decide la implementación
{ provide: POST_REPOSITORY, useClass: PostGraphQLRepository }

```

Cambiar el backend (GraphQL ↔ Supabase) es cambiar una línea en `app.config.ts`.

Signal Stores (gestión de estado)

```
@Injectable({ providedIn: 'root' })
export class FeedStore {
  // Estado privado
  private _posts = signal<Post[]>([]);
  private _loading = signal(false);

  // Estado público (readonly)
  readonly posts = this._posts.asReadonly();
  readonly loading = this._loading.asReadonly();

  // Estado derivado
  readonly isEmpty = computed(() => this._posts().length === 0);

  async loadFeed() {
    this._loading.set(true);
    const data = await this.postService.getFeed();
    this._posts.set(data);
    this._loading.set(false);
  }
}
```

Internacionalización (i18n) — 14 idiomas

```
flowchart LR
    User["Usuario\ncambia idioma"]
    TS["TranslationService\ncurrentLang Signal"]
    LS["localStorage\nsavorealo_lang"]
    Pipe["TranslatePipe\n{{ 'key' | translate }}"]
    Components["Componentes\nre-renderizan"]

    User --> TS
    TS --> LS
    TS --> Pipe
    Pipe --> Components
```

Idiomas soportados

Código	Idioma	Regiones
es	Español	España, México
en	English	EEUU, UK, Canadá, Australia, Singapur
fr	Français	Francia, Suiza
de	Deutsch	Alemania, Suiza
it	Italiano	Italia
pt-BR	Português	Brasil
nl	Nederlands	Países Bajos
sv	Svenska	Suecia
no	Norsk	Noruega
ja	日本語	Japón
ko	한국어	Corea del Sur
ar	العربية	Emiratos Árabes
hi	हिन्दी	India
id	Bahasa Indonesia	Indonesia

Uso en código

```
// En componente
this.translationService.currentLang() // → 'es' | 'en' | 'ja' | ...
this.translationService.setLanguage('ja') // persiste en localStorage
this.translationService.translate('auth.login_btn') // → 'ログイン'
```

```
←!— En templates →  
{{ 'shell.explore' | translate }}  
{{ 'auth.register.create_btn' | translate }}
```

Añadir nuevas claves

Añadir la clave en los **14 bloques** de `translation.service.ts`:

```
// En cada uno de los 14 bloques (es, en, fr, de, it, pt-BR, nl, sv, no, ja, ko, etc.)  
'mi_seccion.mi_clave': 'Texto en el idioma correspondiente',
```

Traducción de contenido dinámico

Las descripciones de posts y los mensajes de chat se traducen en tiempo real.

```

sequenceDiagram
    actor U as Usuario
    participant CT as ContentTranslationService
    participant Cache as Caché (Map)
    participant API as MyMemory API

    U->>CT: translate("Ciao mondo")
    CT->>Cache: ¿Existe "en::Ciao mondo"?
    alt En caché
        Cache-->>CT: "Hello world"
    else Sin caché
        CT->>API: GET /get?q=Ciao mondo&langpair=auto|en
        API-->>CT: { responseData: { translatedText: "Hello world" } }
        CT->>Cache: Guarda "en::Ciao mondo" → "Hello world"
    end
    CT-->>U: "Hello world"

```

MyMemory API: gratuita, sin API key, límite 5.000 palabras/día por IP. Soporta RTL para árabe (`dir="rtl"`).

SSR — Guarda de APIs de navegador

```

// ✅ Correcto – protege código browser-only
constructor(
  @Inject(PLATFORM_ID) private platformId: object
) {}

someMethod() {
  if (isPlatformBrowser(this.platformId)) {
    localStorage.setItem('key', 'value'); // solo en browser
  }
}

```

Variables de entorno

`src/environments/environment.ts`:

```
export const environment = {  
  production: false,  
  supabaseUrl: 'https://<project-ref>.supabase.co',  
  supabaseKey: '<supabase-anon-key>',  
  apiUrl: 'http://localhost:8787',    // ← local  
  // apiUrl: 'https://develop.app.savorealo.com/api', // ← producción  
}
```

Backend — API GraphQL

API GraphQL desplegada en el edge con **Cloudflare Workers**, schema code-first con **Pothos**, ORM **Prisma 7** sobre **PostgreSQL/Supabase** y autenticación con JWT de Supabase verificado mediante HMAC-SHA256.

Comandos principales

```
npm install
npx prisma generate      # Genera el cliente Prisma
npx wrangler dev         # Worker local → http://localhost:8787/graphql
npm run dev              # tsx watch (sin Wrangler)
npm test                 # Tests con Vitest
npm run deploy           # Deploy a Cloudflare Workers
npm run cf-typegen       # Regenera tipos de Cloudflare
```

Stack

Capa	Tecnología	Detalle
Runtime	Cloudflare Workers	Edge, serverless, global
API	GraphQL Yoga	Server GraphQL compatible Workers
Schema	Pothos	Code-first, plugins Prisma + Relay
ORM	Prisma 7	Type-safe, con Prisma Accelerate
Base de datos	PostgreSQL / Supabase	Managed PostgreSQL
Auth	JWT HMAC-SHA256	Tokens emitidos por Supabase Auth
Testing	Vitest	Unitario e integración

Arquitectura

```
flowchart TD
    Client["Cliente\n(Angular / Android)"]

    subgraph Worker ["Cloudflare Worker"]
        Entry["src/index.ts\nCORS · Routing · Auth middleware"]
        GQL["GraphQL Yoga\n+ Pothos Schema"]
        Auth["src/lib/auth.ts\nverifyJWT HMAC-SHA256"]
        Errors["src/lib/errors.ts\nTraduce errores BD"]
    end

    subgraph Schema ["src/schema/ (Pothos)"]
        UserS["user/"]
        PostS["post/"]
        CommentsS["comment/"]
        SocialS["social/"]
        RecipeS["recipe/"]
        SearchS["search/"]
    end
    end

    subgraph Data ["Datos"]
        Prisma["Prisma 7\n+ Accelerate"]
        PG["PostgreSQL\nSupabase"]
        SupaAuth["Supabase Auth\n(verifica JWT)"]
    end
    end

    Client -->|"POST /graphql\nAuthorization: Bearer JWT"| Entry
    Entry --> Auth
    Auth -->|"userId extraído"| GQL
    GQL --> Schema
    Schema --> Prisma
    Prisma --> PG
    Auth -.→|"Verifica firma"| SupaAuth
```

Estructura de carpetas

```
src/
├─ index.ts          Entry point: CORS, /graphql, /health, contexto, JWT, errores
├─ lib/
│   ├─ auth.ts       Verificación JWT con HMAC-SHA256
│   ├─ prisma.ts      Instancia y configuración de Prisma
│   └─ errors.ts      Clasificación y traducción de errores de BD
├─ schema/           Schema GraphQL code-first con Pothos
│   ├─ user/          Tipos y resolvers de usuario
│   ├─ post/          Feed, posts, media
│   ├─ comment/       Comentarios
│   ├─ social/        Follow, save, like
│   ├─ recipe/        Recetas + generación IA
│   └─ search/        Búsqueda full-text
└─ services/          Servicios de aplicación reutilizables
```

Endpoints

Endpoint	Método	Descripción
<code>/graphql</code>	POST	API GraphQL + playground GraphiQL
<code>/health</code>	GET	<code>{ status: "ok", timestamp: "..."} </code>

Ejemplo curl


```
curl https://develop.app.savorealo.com/api/graphql \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer <token>" \
  -d '{"query":{"feed(first:5){ edges{ node{ id title } } } }"}
```

Autenticación

```
sequenceDiagram
    participant C as Cliente
    participant W as Worker
    participant S as Supabase Auth

    C->>W: POST /graphql + Authorization: Bearer <JWT>
    W->>W: Extrae JWT del header
    W->>W: verifyJWT(token, SUPABASE_JWT_SECRET)
    Note over W: HMAC-SHA256 - sin llamada externa
    W->>W: Extrae { userId, email } del payload
    W-->>C: Respuesta GraphQL (userId en contexto)

    Note over W,S: En producción ENVIRONMENT ≠ development → auth obligatoria
```

En `ENVIRONMENT=development` la autenticación es opcional y se usa un usuario por defecto.

Variables de entorno

`.dev.vars` (Wrangler)

```
DATABASE_URL=prisma+postgres://accelerate.prisma-data.net/?api_key=TU_KEY
SUPABASE_JWT_SECRET=tu-jwt-secret
ENVIRONMENT=development
```

.env (Prisma CLI)

```
DATABASE_URL=postgresql://user:pass@pooler.supabase.com:6543/postgres?pgbouncer
DIRECT_URL=postgresql://user:pass@pooler.supabase.com:5432/postgres
```

Variable	Uso
<code>DATABASE_URL</code>	Conexión del Worker (o Hyperdrive si está configurado)
<code>DIRECT_URL</code>	Conexión directa para Prisma CLI y migraciones
<code>SUPABASE_JWT_SECRET</code>	Secreto HMAC para verificar tokens de Supabase Auth
<code>ENVIRONMENT</code>	En producción exige usuario autenticado en todas las operaciones

Operaciones GraphQL

Usuarios

Operación	Tipo	Descripción
<code>user(id)</code>	Query	Perfil por ID. Respeta privacidad
<code>suggestedUsers(preferenceIds, limit)</code>	Query	Usuarios con preferencias similares
<code>checkUsername(username)</code>	Query	Comprueba disponibilidad
<code>searchUsers(query)</code>	Query	Búsqueda por nombre/username
<code>updateProfile(...)</code>	Mutation	Actualiza perfil personal o de negocio

```
# Obtener perfil
query {
  user(id: "uuid") {
    id username display_name avatar_url user_type
    followers_count following_count posts_count
    isFollowing allergies savedPostsCount
  }
}

# Actualizar perfil (todos los campos opcionales)
mutation {
  updateProfile(username: "nuevo", bio: "Amante de la cocina", location: "Madrid") {
    id username display_name
  }
}
```

Feed y Posts

```

flowchart LR
    subgraph Feeds
        F["feed\nSiguiendo · últimos 7d\nRelay pagination"]
        D["discoverFeed\nPersonalizado\nAlgoritmo scoring"]
        U["userPosts\nGrid de perfil\nOffset pagination"]
    end

    subgraph Mutations
        C["createPost\ntitle · content · imageUrl"]
        Del["deletePost\nsolo propios"]
        L["toggleLike\nnda / quita like"]
        S["toggleSave\nguarda / elimina"]
    end

```

Operación	Tipo	Paginación	Descripción
<code>feed(first, after)</code>	Query	Relay cursores	Posts de seguidos, últimos 7 días
<code>discoverFeed(limit, offset)</code>	Query	Offset	Feed personalizado con scoring
<code>userPosts(userId, limit, offset)</code>	Query	Offset	Grid de perfil de usuario
<code>createPost(title, content, imageUrl)</code>	Mutation	—	<code>post_type</code> auto: PHOTO si hay imagen
<code>deletePost(postId)</code>	Mutation	—	Solo propios
<code>toggleLike(postId)</code>	Mutation	—	Alterna like ↔ no like
<code>toggleSave(postId)</code>	Mutation	—	Alterna guardado ↔ no guardado

```
# Feed con paginación Relay
query {
  feed(first: 10, after: "cursor") {
    edges {
      cursor
      node {
        id title description post_type
        likes_count comments_count saves_count
        liked saved imageUrl created_at
        author { id username avatar_url }
        recipe { name time_required difficulty }
      }
    }
    pageInfo { hasNextPage endCursor }
  }
}
```

Comentarios

Operación	Tipo	Descripción
<code>comments(postId)</code>	Query	Lista comentarios (cronológico)
<code>addComment(postId, text)</code>	Mutation	Añadir comentario
<code>deleteComment(commentId)</code>	Mutation	Eliminar (solo propios)

Social

Operación	Tipo	Descripción
<code>toggleFollow(userId)</code>	Mutation	Seguir / dejar de seguir
<code>respondFollowRequest(requestId, accept)</code>	Mutation	Acepta o rechaza solicitud
<code>notifications(limit, offset)</code>	Query	Notificaciones del usuario
<code>pendingFollowRequests</code>	Query	Solicitudes pendientes de aceptar

`toggleFollow` lanza error `Cannot follow yourself` si el `userId` es el propio.

Recetas e IA

```
sequenceDiagram
    participant U as Usuario
    participant W as Worker
    participant AI as Agente IA (Webhook)
    participant DB as PostgreSQL

    U->>W: mutation generateRecipe(message: "...")
    W->>DB: INSERT ai_generations { status: PENDING }
    W->>AI: Webhook asíncrono (fire-and-forget)
    W-->>U: { started: true }
    Note over W,U: Respuesta inmediata
    AI->>AI: Procesa prompt con LLM
    AI->>DB: UPDATE ai_generations { status: DONE, result: {...} }
    U->>W: query myGenerations
    W->>DB: SELECT WHERE userId AND status = DONE
    W-->>U: Recetas generadas
    U->>W: mutation createRecipePost(generationId)
    W-->>U: Post publicado en feed
```

CAMPOS DE **RECIPE** (ANIDADO EN POSTS)

Campo	Tipo	Descripción
<code>id</code>	ID	UUID
<code>name</code>	String	Nombre de la receta
<code>description</code>	String?	Descripción libre
<code>steps</code>	String	JSON stringificado con los pasos
<code>time_required</code>	Int?	Minutos de preparación
<code>servings</code>	Int?	Número de porciones
<code>difficulty</code>	String	<code>EASY</code> · <code>MEDIUM</code> · <code>HARD</code>
<code>estimated_cost</code>	String?	Coste estimado

Búsqueda

Operación	Tipo	Motor	Descripción
<code>searchPosts(query, limit, offset)</code>	Query	<code>tsvector</code> PostgreSQL	Full-text en título y descripción
<code>searchRecipes(query, limit, offset)</code>	Query	<code>search_vector</code>	Full-text en tabla <code>recipes</code>
<code>searchUsers(query)</code>	Query	ILIKE	Por nombre y username

La búsqueda full-text usa diccionario **español** y ordena por relevancia `ts_rank` .

Enums de la base de datos

Enum	Valores
UserType	PERSON · RESTAURANT · BAR
PostType	PHOTO · VIDEO · TEXT · RECIPE
DifficultyLevel	EASY · MEDIUM · HARD
NotificationType	LIKE · COMMENT · FOLLOW · MENTION · RECIPE_SAVE
ConversationType	DIRECT · GROUP
AiGenerationStatus	PENDING · DONE · FAILED
PostCategory	TRENDING · ITALIAN · MEXICAN · JAPANESE · CHINESE · DESSERTS · VEGAN · QUICK_EASY · BURGER · SEAFOOD · COCKTAILS · BREAKFAST · LUNCH · DINNER · SNACKS · HEALTHY · COMFORT_FOOD · STREET_FOOD

Tablas pendientes de implementar

Tabla	Descripción	Prioridad
<code>user_settings</code>	Privacidad, idioma, tema, notificaciones	Alta
<code>notifications</code>	Like, comment, follow, mention	Alta
<code>saved_posts</code> (query)	Listar guardados (toggleSave ya existe)	Alta
<code>viewed_posts</code>	Tracking para discover feed	Alta
<code>stories</code> / <code>viewed_stories</code>	Stories temporales con expiración	Media
<code>conversations</code> / <code>direct_messages</code>	Mensajería directa y grupal	Media
<code>places</code> / <code>place_reviews</code>	Restaurantes, bares y reseñas	Media
<code>ingredients</code> / <code>recipe_ingredients</code>	Ingredientes detallados	Media
<code>contacts</code>	Lista de contactos	Baja

Manejo de errores

`src/lib/errors.ts` intercepta errores de Prisma/PostgreSQL y los transforma en errores GraphQL controlados. Nunca se exponen mensajes técnicos de base de datos al cliente.

```
// Ejemplo: unique constraint → mensaje amigable
PrismaClientKnownRequestError P2002 → GraphQLError("USERNAME_TAKEN")
PrismaClientKnownRequestError P2025 → GraphQLError("NOT_FOUND")
```

Mobile Android — Kotlin

App Android nativa con **Kotlin 2.2** y **Jetpack Compose**, arquitectura limpia con separación Remote/Repository, **Hilt** para inyección de dependencias, **Room** para persistencia local y **Supabase KT** para autenticación y tiempo real.

Comandos principales

```
./gradlew installDebug      # Instala en dispositivo o emulador
./gradlew assembleDebug    # Genera APK debug
./gradlew assembleRelease  # Genera APK/AAB release
./gradlew test              # Tests unitarios (JUnit)
./gradlew connectedAndroidTest # Tests instrumentados (Espresso)
./gradlew tasks              # Muestra todas las tareas
```

También se puede abrir en **Android Studio** y ejecutar la configuración `app` directamente.

Stack

Capa	Tecnología	Detalle
Lenguaje	Kotlin 2.2	Coroutines, Flow, KSP
UI	Jetpack Compose	Declarativa, Material 3
Navegación	Navigation Compose	Type-safe routes
DI	Hilt + KSP	Inyección de dependencias
Persistencia local	Room	Caché offline, generaciones IA
Auth / Realtime	Supabase KT	Auth, Storage, Realtime
API GraphQL	Retrofit + OkHttp	Cliente HTTP, tipado con data classes
Cámara	CameraX	Captura de fotos para posts
Tareas diferidas	WorkManager	Recordatorios, subida diferida
SDK mínimo	minSdk 34	Android 14+
SDK objetivo	targetSdk / compileSdk 36	Android 16

Arquitectura

```
flowchart TD
    subgraph UI ["Capa UI (ui/)"]
        Screens["Screens\n(Jetpack Compose)"]
        ViewModels["ViewModels\n(StateFlow / UiState)"]
        Nav["Navigation\n(Routes.kt + NavGraph)"]
    end

    subgraph Domain ["Capa de Dominio"]
        Repos["Repositories\n(lógica de negocio)"]
        Models["Domain Models\n(Post, User, Recipe...)"]
    end

    subgraph Data ["Capa de Datos (data/)"]
        Remote["Remote Data Sources\n(Supabase KT + Retrofit/GraphQL)"]
        Local["Local Data Sources\n(Room DAOs)"]
    end

    subgraph DI ["Inyección de Dependencias (di/)"]
        Hilt["Hilt Modules\n(Network · Database · Supabase · App)"]
    end

    subgraph External ["Externo"]
        SB["Supabase\nAuth · Storage · Realtime"]
        GQL["API GraphQL\n(Cloudflare Worker)"]
    end

    Screens --> ViewModels
    ViewModels --> Repos
    Repos --> Remote
    Repos --> Local
    Remote --> SB
    Remote --> GQL
    Hilt -.→|"provee"| Remote
    Hilt -.→|"provee"| Local
    Hilt -.→|"provee"| Repos
```

Patrón Remote vs Repository

Remote Data Source — "Habla con la API"

Solo hace peticiones. Sin lógica de negocio. Devuelve exactamente lo que recibe.

```
class PostRemoteDataSource @Inject constructor(
    private val supabase: SupabaseClient,
    private val graphqlService: GraphQLService
) {
    // Solo pregunta a Supabase/GraphQL
    suspend fun getPostById(id: String): PostDto =
        supabase.from("posts").select().eq("id", id).decodeSingle()

    suspend fun getFeed(first: Int, after: String?): FeedConnection =
        graphqlService.query(FeedQuery(first, after))
}
```

Repository — "Tiene lógica de negocio"

Combina remote + local, aplica reglas, maneja errores.

```
class PostRepository @Inject constructor(
    private val remote: PostRemoteDataSource,
    private val localDao: PostDao
) {
    // Lógica de negocio: caché offline, errores controlados, transformación
    suspend fun getPost(id: String): Result<Post> = runCatching {
        localDao.getById(id)
        ?: remote.getPostById(id).toDomain().also { localDao.insert(it.toEntity()) }
    }

    fun getFeedPager(): Flow<PagingData<Post>> = Pager(
        config = PagingConfig(pageSize = 10)
    ) { FeedPagingSource(remote) }.flow
}
```

Regla: los ViewModels solo conocen Repositories. Los Repositories son los únicos que conocen Remote y Local.

Estructura del proyecto

```
app/src/main/java/es/PapayaSA/letmecook/
├─ data/
│   ├─ model/
│   │   └─ Post.kt, User.kt, Recipe.kt, Message.kt...    ← Domain models
│   │   └─ dto/    PostDto.kt, UserDto.kt...            ← DTOs de red
│   └─ remote/
│       └─ PostRemoteDataSource.kt
│       └─ UserRemoteDataSource.kt
│       └─ GraphQLService.kt                            ← Interface Retrofit
│   └─ repository/
│       └─ PostRepository.kt
│       └─ UserRepository.kt
│       └─ RecipeRepository.kt
│   └─ local/
│       └─ AppDatabase.kt                                ← Room DB
│       └─ dao/    PostDao.kt, RecipeDao.kt...
├─ di/
│   └─ SupabaseModule.kt    ← SupabaseClient singleton
│   └─ NetworkModule.kt     ← Retrofit + OkHttp singleton
│   └─ DatabaseModule.kt    ← Room DB + DAOs
│   └─ AppModule.kt        ← Repositorios y otros
├─ ui/
│   └─ navigation/
│       └─ Routes.kt        ← Definición de rutas
│       └─ NavGraph.kt      ← Grafo de navegación Compose
│   └─ screens/             ← Una carpeta por pantalla
│       └─ home/    HomeScreen.kt + HomeViewModel.kt
│       └─ auth/    LoginScreen.kt + RegisterScreen.kt
│       └─ profile/  ProfileScreen.kt + ProfileViewModel.kt
│       └─ ...
│   └─ theme/
│       └─ Color.kt, Typography.kt, Shape.kt
│       └─ Theme.kt        ← MaterialTheme
│   └─ components/         ← Composables reutilizables
```

```

├─ utils/
|   ├─ Constants.kt      ← URL base, claves
|   ├─ ImageUtils.kt
|   └─ LanguageUtils.kt
└─ worker/
    ├─ ReminderWorker.kt  ← Recordatorios de recetas
    └─ PostUploadWorker.kt ← Subida diferida de posts

```

Pantallas y navegación

```

flowchart TD
    Login --> Register
    Login --> CompleteProfile
    Login --> Home

    Home --> PostDetail
    Home --> UserProfile
    Home --> Camera

    subgraph Main ["Navegación principal (Bottom Bar)"]
        Home["Home\nFeed principal"]
        Explore["Explore\nDescubrir posts"]
        Generator["Generator\nRecetas IA"]
        Messages["Messages\nConversaciones"]
        Profile["Profile\nPerfil propio"]
    end

    Explore --> PostDetail
    Explore --> UserProfile
    Profile --> Settings
    Profile --> Following
    Profile --> Followers
    Messages --> Conversation

```


Ruta Compose	Pantalla	Descripción
Login	Inicio de sesión	Email / contraseña / Google
Register	Registro	Alta de cuenta nueva
CompleteProfile	Completar perfil	Tras alta o login social
Home	Feed principal	Posts de usuarios seguidos
Explore	Explorar	Descubrir posts y usuarios
Generator	Generador IA	Generar recetas con IA
Profile	Perfil propio	Posts, bio, seguidores
UserProfile(userId)	Perfil ajeno	Perfil público
Camera(mode)	Cámara	CameraX para fotos de posts
PostDetail(postId)	Detalle post	Receta completa, comentarios
Notifications	Notificaciones	Likes, follows, menciones
Messages(pendingUserId)	Mensajes	Lista de conversaciones
Conversation(conversationId, otherUserId, otherUserName)	Chat	Conversación directa
Following(userId)	Siguiendo	Lista de seguidos
Followers(userId)	Seguidores	Lista de seguidores

Módulos Hilt

```
// SupabaseModule.kt
@Module @InstallIn(SingletonComponent::class)
object SupabaseModule {
    @Provides @Singleton
    fun provideSupabase(): SupabaseClient = createSupabaseClient(
        supabaseUrl = BuildConfig.SUPABASE_URL,
        supabaseKey = BuildConfig.SUPABASE_ANON_KEY
    ) {
        install(Auth)
        install(Realtime)
        install(Storage)
    }
}

// NetworkModule.kt
@Module @InstallIn(SingletonComponent::class)
object NetworkModule {
    @Provides @Singleton
    fun provideRetrofit(): Retrofit = Retrofit.Builder()
        .baseUrl(Constants.URL) // "https://develop.app.savorealo.com/api/"
        .addConverterFactory(GsonConverterFactory.create())
        .client(OkHttpClient.Builder()
            .addInterceptor(AuthInterceptor()) // añade JWT
            .build())
        .build()
}
```

Configuración de URLs

`utils/Constants.kt` :

```
// Producción
const val URL = "https://develop.app.savorealo.com/api/"

// Backend local en emulador Android
const val URL = "http://10.0.2.2:8787/"

// Backend local en dispositivo físico (IP LAN del PC)
const val URL = "http://192.168.1.XX:8787/"
```

Retrofit construye el endpoint final: `${URL}graphql`

Workers (tareas en background)

Worker	Trigger	Descripción
<code>ReminderWorker</code>	<code>WorkManager</code> periódico	Notifica cuando hay recetas guardadas
<code>PostUploadWorker</code>	<code>WorkManager</code> + constraints de red	Sube posts cuando hay conectividad

```
// Enqueue con constraint de red
val uploadRequest = OneTimeWorkRequestBuilder<PostUploadWorker>()
    .setConstraints(
        Constraints.Builder().setRequiredNetworkType(NetworkType.CONNECTED).build()
    )
    .setInputData(workDataOf("postId" to postId))
    .build()
WorkManager.getInstance(context).enqueue(uploadRequest)
```

Build y despliegue

```
./gradlew assembleDebug    # APK debug (development)
./gradlew assembleRelease  # APK/AAB release (requiere keystore)
```

Configuración en `app/build.gradle.kts`:

Parámetro	Valor
<code>applicationId</code>	<code>es.PapayaSA.letmecook</code>
<code>compileSdk</code>	36
<code>minSdk</code>	34 (Android 14)
<code>targetSdk</code>	36

Antes de publicar: configurar firma con keystore, mover claves de Supabase/Google a `secret.properties` o BuildConfig secrets, revisar reglas ProGuard/R8.

Flujos del sistema

Esta sección documenta los flujos más importantes de Savorealo: autenticación, ciclo de vida de una request, generación de recetas con IA, feeds, mensajería en tiempo real, subida de imágenes y CI/CD.

1. Flujo de autenticación

```
sequenceDiagram
    actor U as Usuario
    participant C as Cliente (Web / Mobile)
    participant S as Supabase Auth
    participant W as Worker GraphQL
    participant DB as PostgreSQL

    U->>C: Introduce email + contraseña (o Google Sign-In)
    C->>S: supabase.auth.signInWithPassword(...)
    S->>S: Valida credenciales
    S-->>C: { access_token (JWT), refresh_token }
    Note over C: Web: guarda en localStorage<br/>Mobile: persistencia Supabase

    C->>W: POST /graphql\nAuthorization: Bearer JWT
    W->>W: verifyJWT(token, SUPABASE_JWT_SECRET)\nHMAC-SHA256 - sin llamada ex
    W->>W: Extrae { userId, email } del payload
    W->>DB: Prisma query/mutation con userId en contexto
    DB-->>W: Datos
    W-->>C: Respuesta GraphQL
```

Renovación automática del token

```
flowchart LR
    Token["JWT expira\n(por defecto 1h)"]
    SDK["Supabase SDK\n(JS o KT)"]
    Refresh["POST /auth/v1/token\ngrant_type=refresh_token"]
    New["Nuevo JWT"]
    Retry["Reintenta request original"]

    Token --> SDK
    SDK --> Refresh
    Refresh --> New
    New --> Retry
```

```
Token → SDK
SDK → Refresh
Refresh → New
New → Retry
```

2. Flujo de request completo

```
flowchart TD
    C["Cliente\nAngular / Android"]

    subgraph Edge ["Cloudflare Edge (Worker)"]
        CORS["Validación CORS\nOrigin permitido?"]
        Route["Routing"]
        Health["/health → JSON status"]
        Auth["Auth Middleware\nverifyJWT()"]
        Ctx["Contexto GraphQL\n{ prisma, userId }"]
        Resolver["Pothos Resolver\nquery / mutation"]
        ErrH["Error Handler\nerrors.ts → mensaje amigable"]
    end

    subgraph Data ["Datos"]
        Acc["Prisma Accelerate\n(connection pooling)"]
        PG["PostgreSQL\nSupabase"]
    end

    C -->|POST /graphql + Bearer JWT| CORS
    CORS --> Route
    Route -->|/health| Health
    Route -->|/graphql| Auth
    Auth --> Ctx
    Ctx --> Resolver
    Resolver --> Acc --> PG
    PG --> Acc --> Resolver
    Resolver --> C
    Resolver -->|error| ErrH --> C
```

3. Flujo de generación de recetas con IA

```
sequenceDiagram
    actor U as Usuario
    participant C as Cliente
    participant W as Worker GraphQL
    participant DB as PostgreSQL
    participant AI as Agente IA (Webhook externo)

    U->>C: Escribe prompt: "Paella para 4 personas"
    C->>W: mutation generateRecipe(message: "...")
    W->>DB: INSERT ai_generations { userId, prompt, status: PENDING }
    W->>AI: Webhook asíncrono (fire-and-forget)
    W-->>C: { started: true }
    Note over C,W: Respuesta inmediata - el cliente no espera

    loop Polling (query myGenerations)
        C->>W: query myGenerations
        W->>DB: SELECT WHERE userId AND status IN (PENDING, DONE)
        W-->>C: [{ id, status: PENDING, ... }]
    end

    AI->>AI: Procesa con LLM
    AI->>DB: UPDATE ai_generations SET status = DONE, result = { pasos, ingred:

    C->>W: query myGenerations
    W-->>C: [{ id, status: DONE, post: null }]
    U->>C: Acepta receta generada
    C->>W: mutation createRecipePost(generationId)
    W->>DB: INSERT posts + recipes + UPDATE ai_generations.converted_to_post_id
    W-->>C: Post publicado
    Note over C: Post aparece en el feed
```

Estados de generación

Estado	Icono	Descripción
PENDING	Procesando	El agente IA está generando la receta
DONE	Listo	Receta generada, disponible para publicar
FAILED	Error	Falló la generación — se puede reintentar

4. Flujo del feed principal (Following)

```

flowchart TD
    Open["Usuario abre el Feed"]
    
    subgraph Query ["query feed(first: 10)"]
        Filter["WHERE author_id IN following_ids\nAND created_at > NOW() - 7 d"]
        Order["ORDER BY created_at DESC"]
        Page["LIMIT 10 AFTER cursor"]
    end
    
    Response["edges[] + pageInfo\n{ hasNextPage, endCursor }"]
    Scroll["Usuario hace scroll hasta el final"]
    Next["query feed(first: 10, after: endCursor)"]

    Open --> Filter
    Filter --> Order
    Order --> Page
    Page --> Response
    Response --> Scroll
    Scroll --> Next
    Next --> Filter

```

- Paginación **Relay** con cursores (no offset) — evita posts duplicados/saltados
- `liked` y `saved` devueltos por post: el cliente sabe el estado sin queries extra
- `imageUrl`: primera imagen del post (`position: 0` en `post_media`)

5. Flujo del feed Descubrir (algoritmo de scoring)

```
flowchart LR
    subgraph RPC ["PostgreSQL RPC: get_personalized_feed(userId, limit, offset)"]
        E["Engagement score\nlikes + comments + saves"]
        A["Affinity score\npreferencias culinarias del usuario"]
        F["Freshness score\npenalización por antigüedad"]
        Score["score = E + A + F"]
    end
    Query["query discoverFeed(limit: 20, offset: 0)"]
    Result["Posts ordenados por score DESC"]
```

```
Query → RPC
E → Score
A → Score
F → Score
Score → Result
```

6. Flujo de mensajería en tiempo real

```
sequenceDiagram
    participant A as Usuario A (remitente)
    participant SB as Supabase Realtime
    participant DB as PostgreSQL
    participant B as Usuario B (receptor)

    Note over A,B: Ambos suscritos al canal de la conversación via WebSocket
    A->>SB: INSERT direct_messages { conversation_id, sender_id, content }
    SB->>DB: Persiste el mensaje
    SB-->>B: Evento Realtime: nuevo mensaje
    Note over B: Recibe el mensaje instantáneamente sin polling
    B->>SB: INSERT direct_messages { content: "respuesta" }
    SB-->>A: Evento Realtime: respuesta recibida
```

La mensajería **no pasa por el Worker GraphQL** — usa Supabase Realtime directamente para latencia mínima.

7. Flujo de subida de imagen y creación de post

```
sequenceDiagram
    actor U as Usuario
    participant C as Cliente (Web/Mobile)
    participant SS as Supabase Storage
    participant W as Worker GraphQL
    participant DB as PostgreSQL

    U->>C: Selecciona/captura imagen
    Note over C: Web: FileUpload → ArrayBuffer<br/>Mobile: CameraX → Bitmap
    C->>SS: upload(bucket, path, imageData)
    SS-->>C: URL pública: https://...supabase.co/storage/v1/object/public/...
    C->>W: mutation createPost(title, content, imageUrl)
    W->>DB: INSERT posts { title, content, post_type: PHOTO }
    W->>DB: INSERT post_media { media_url, media_type: IMAGE, position: 0 }
    W-->>C: Post creado con ID
    Note over C: Post aparece en el feed de seguidores
```

8. Flujo de traducción de contenido

```
sequenceDiagram
    actor U as Usuario
    participant C as Cliente
    participant CTS as ContentTranslationService
    participant Cache as Caché en memoria (Map)
    participant API as MyMemory API (pública)

    U->>C: Clic en [Traducir]
    C->>CTS: translate("Ciao mondo")
    CTS->>Cache: ¿Existe clave "en::Ciao mondo"?
        alt Caché HIT
            Cache-->>CTS: "Hello world"
        else Caché MISS
            CTS->>API: GET /get?q=Ciao mondo&langpair=auto|en
            Note over API: Sin API key · Detección automática del idioma origen
            API-->>CTS: { translatedText: "Hello world" }
            CTS->>Cache: Guarda "en::Ciao mondo" → "Hello world"
        end
        CTS-->>C: "Hello world"
    C->>C: Renderiza texto traducido
    Note over C: Si idioma activo = 'ar' → aplica dir="rtl" al contenedor
```

9. Flujo de CI/CD

```
flowchart TD
    Push["git push → GitHub"]

    subgraph CI ["GitHub Actions"]
        subgraph FE ["Frontend (Angular)"]
            FI["npm ci"]
            FL["npm run lint"]
            FT["npm test (Vitest)"]
            FB["npm run build:cloudflare"]
            FD["Deploy → Cloudflare Pages"]
            FI → FL → FT → FB → FD
        end
    end

    subgraph BE ["Backend (Worker)"]
        BI["npm ci"]
        BT["npm test (Vitest)"]
        BD["npx wrangler deploy"]
        BI → BT → BD
    end

    subgraph MOB ["Mobile Android"]
        MT["./gradlew test"]
        MB["./gradlew assembleRelease"]
        MA["Artefacto APK/AAB"]
        MT → MB → MA
    end

    Push → FI
    Push → BI
    Push → MT
```

Resumen de flujos

Flujo	Protocolos	Latencia típica
Autenticación	HTTPS + Supabase Auth	~300ms
Request GraphQL	HTTPS + JWT	~100ms (edge)
Generación IA	GraphQL (fire-and-forget) + Webhook	5–30s async
Feed following	GraphQL + Prisma + PostgreSQL	~150ms
Feed descubrir	GraphQL + PostgreSQL RPC	~200ms
Mensajería	WebSocket (Supabase Realtime)	<100ms
Subida imagen	HTTPS multipart → Supabase Storage	~500ms–2s
Traducción	HTTPS → MyMemory API + caché local	<50ms (caché) / ~400ms (API)